

Game search algorithms: Minimax with Alpha-Beta Pruning

The core search algorithm is minimax with alpha-beta pruning. By analysing the game, it is observed that Cascade game is deterministic and has perfect information. Given these properties, minimax is well-suited for selecting the optimal move as it assumes optimal play from both players and evaluates the best achievable payoff under worst case opposition.

Minimax evaluates board states using a heuristic evaluation function, allowing the agent to estimate the quality of positions without needing to play the game until terminal state, unlike Monte Carlo Tree Search approaches that rely on simulation-based rollouts. Compared to greedy search, which only considers the immediate best move, minimax enables deeper lookahead by exploring future game states. This allows the agent to account for potential threats and opportunities several moves ahead when selecting the most appropriate action, resulting in more robust and strategically informed decisions.

Purely rely on minimax can be computationally expensive because it explores all possible actions and responses in the search tree. Since the 8×8 board can contain many tokens with multiple legal moves, the number of possible states grows rapidly as the search depth increases. Given the time constraints, it is infeasible to evaluate every possible move exhaustively. Therefore, pruning strategies are necessary to improve efficiency. Alpha-beta pruning was implemented to reduce the number of nodes evaluated in the search tree. It skips branches that cannot influence final decision while still preserving the optimality of minimax. As a result, it significantly reduces the time taken to search each layer without comprising the decision quality. To further improve the pruning strategy, each possible actions on the current board state is then sorted with a simple heuristic to improve the move order.

Having a strong and computationally efficient evaluation function is important in minimax, as it incorporates the domain knowledge to guide the search process. This contrasts with Monte Carlo Search Tree, which relies on random simulations to evaluate actions rather than explicitly using a customised evaluation function while expanding the tree. The Cascade game consists of two distinct phases, which are placement phase and play phase. Since each phase has different constraints and decision spaces, two different heuristic functions were designed to evaluate states appropriately for each phase.

Placement Phase Evaluation Function

During the placement phase, the evaluation function is designed as a rule-based scoring system that estimates positional advantage by summing multiple heuristic function components for both agent's and the enemy's stacks. Instead of using explicit tuneable weights, each heuristic contributes through its inherent score magnitude and structure, meaning that importance is encoded implicitly through how strongly each feature affects the final score. The function focuses on long-term positional advantage rather than immediate tactical outcomes, since only the Place action is available. Therefore, it prioritises structural stability, board control, and future interaction potential through combined evaluation of both friendly and enemy stacks.

Centrality and positional stability are calculated using a precomputed distance heatmap, where each board position is assigned a value based on its distance from both the centre and the edges. Positions closer to the centre are generally preferred, as they provide greater flexibility and reduce the risk of being pushed off the board through cascade action. This is particularly relevant given that all stacks initially have equal height, and the board size is 8×8 , making central positions strategically safer. In contrast, edge and corner positions are associated with lower score, as they reduce mobility and increase the risk of being pushed off the board through cascade action.

Besides that, cascade push-off risk is also part of the evaluation function, as early instability can lead to irreversible losses. It is determined by checking whether nearby enemy stacks could potentially trigger a cascade that would push the agent's stack off the board. By penalising such dangerous placements, it reduces the likelihood of early token loss and encourages safer, more stable positioning.

Potential to perform Eat action on the enemy is included to reflect the future elimination in later stages. Although tokens cannot be placed adjacent to enemies after the initial placement phase, minimising the distance to opponent stacks remain strategically beneficial, as it increases the likelihood to eliminate them once the Play phase starts. Since all stacks begin at height 3 and the Eat action requires at least equal height, positioning closer to enemy tokens increases the chances to successfully execute an Eat action. Therefore, a distance-based scoring system is used to reward positions that are closer to enemy tokens and thus more likely to result in eliminating opponents.

Finally, merging potential between friendly stacks is considered by rewarding tokens that are placed adjacent to each other. Forming higher stacks can be advantageous, as they are harder to eliminate through direct Eat actions and perform greater influence

through Cascade action without requiring getting close to the enemy. Moreover, excessive clustering of same coloured token is penalised, as it can create predictable structures that are vulnerable to large-scale cascade attacks, potentially allowing the opponent to push multiple tokens off the board in a single move.

Overall, the evaluation function behaves as an approximation of zero-sum system, where the final score is effectively derived from the difference between the utility of the players side and enemy side. An advantage of the position on the agent side corresponds directly to disadvantages in opponent's potential benefits. This helps to ensure that the agents optimise for relative advantage by considering both its own and the opponents token rather than only evaluating the general board state, which is consistent with the competitive structure of the game and aligns naturally with minimax-based decision-making framework.

Play Phase Evaluation Function

Each move in play phase is assessed using a combination of heuristic functions that are applied depending on the relevant board conditions. The main evaluation function considers the properties of both the current player (the agent) and the opposing player under a zero-sum assumption. This means that a move is not judged solely by how much it benefits the agent, but also by how much it benefits or disadvantage the opponent. For example, a move may appear beneficial for the agent in isolation, but it can still be disadvantageous if it enables the opponent to eliminate, cascade or trap it in the following turn. As a result, such moves are penalised, as the evaluation considers not only immediate gain but also the opponent's potential counterplay. Therefore, the evaluation function aims to maximise the agent's advantage while simultaneously reducing the opponent's opportunities.

To improve efficiency, the positions of both agent and enemy stacks are stored in a dictionary to avoid repeated traversal of the board state, reducing redundant conditions checking and computation during evaluation. The list below provides a brief overview of the factors considered in the main evaluation function.

Properties:

1. Total heights of the tokens
2. Mobility of tokens, estimating through how trapped or free a token is
3. Potential to perform an Eat action against enemy tokens, where condition is checked as $height_{agent} \geq height_{enemy}$
4. Bonus scoring for safe Eat actions (*safe_eat* in code), where additional reward is given when $height_{agent} > height_{enemy}$, as this ensures the enemy cannot immediately eliminate the token with an Eat action
5. Threat assessment, evaluating whether enemy tokens can perform an Eat action on the agent's stacks
6. Cascade benefit of pushing an enemy token
7. Cascade loss risk for the agent as it may lose its own tokens to perform Cascade

The first property is material advantage which is calculated by subtracting the opponent's total height from the agent's total stack height. This property has a high weight of 100 in the final scoring because preserving the agent's overall height while reducing the opponent's directly contributes to long-term advantage and overall winning potential. However, this property alone is insufficient, as it does not account for possible future interactions or threats, and must therefore be combined with other heuristic. Second property evaluates mobility by make sure that agent have room to move and not trapped by the enemy or in the edge. The is measured by counting the number of trapped stacks, where a stack is considered trapped if it has no legal moves available. This value is weighted by 6 and subtracted from the total score because trapped stacks reduce flexibility and limit future action options, making them disadvantageous for the agent.

The third and fourth properties evaluate the agent's eating opportunities. If the agent's token is greater or equals to the enemy token ($height_{agent} \geq height_{enemy}$), the opponent height will be added to the third property while if the token is strictly greater than ($height_{agent} > height_{enemy}$), it will be added to the fourth property. This is because strictly stronger stack is generally a safer eating opportunity than an equal height, whereas equal height eating opportunity still carry risk, as the opponent may also be able to perform an Eat action. Both values are then compared against the opponent eating opportunities so that the agent prefers positions where it can maximise the agent's benefits on eliminating an opponent while limiting the opponent's ability to eat the agent's token. In the total scoring, it will be multiplied by a weight of 50 where a positive difference means the agent has higher benefits from Eat action compared to opponent performing Eat action on the agent and vice versa for negative difference. The fifth property measures how many stacks are at risk of being captured by enemy through Eat action. If an opponent's stack is greater than or equal to the agent's stack, the agent's height will be added to the threat. The score is calculated using $opp_threat - agent_threat$ then weighted by 20 so the agent is rewarded when the opponent is more vulnerable and penalised when the agent own token is under threat.

The last two properties are taken account when the tokens do a cascade movement, which is triggered only when the token height is greater or equal to 2 ($height_{agent} \geq 2$). These properties are included because Cascade interactions can significantly affect long-term material advantage. It will first check the amount of token lost through out of bound by the agent itself if Cascade action was performed. If this happens, the number of lost tokens is added to *agent_cascade_self_loss*, because losing agents' own tokens is undesirable for the agent. Next, the function will check if performing Cascade action pushed an enemy token out of the board. If it is successful, the height of removed opponent's stack will be added to *agent_cascade_kill* which rewards it. Lastly, it also checks whether Cascade pushed any agent's token out of the board or closer to the edge. If it gets pushed out of the board the height is used while for edges, it will get a penalty of 1, as edges are less stable than central position but still preferable to being fully removed from the board. In the final evaluation, cascade kills are rewarded with weight 20 and cascade self-loss is penalised with weight 15, a higher weight is given when cascade kills as it is closer to the end goal.

These properties are then given a score and will be weighted for the final scoring. An additional heuristic function for "endgame" preparation is also created. This is triggered when our opponent has 4 or less token and agent total height is greater than the opponent total height. The endgame property ensures that the agent is chasing and trapping (by surrounding it with agents' token or pushing it to the edge) the enemy token aggressively, it will also eat whenever there is opportunity.

The endgame heuristic function evaluates the number of blocked and free sides around the enemy and rewards the positions where the opponent has fewer escape routes and penalising positions where the opponent can move freely, which is the trap strategy. In the chasing feature, it measures the distance by rewarding the closest and the second-closest agent tokens as well as stronger nearby "hunter" tokens that can potentially capture the opponent. Immediate Eat action opportunities are given a very high reward (weighted by 300) while situations where the agent's token can be captured by the enemy will be penalised. Additionally, it also checks for cascade alignment in the same row or column and see if it is possible to push the opponent out of the board which will be given a high reward if it is possible. This helps the agent to chase and trap better in the endgame instead of wandering or repeating unproductive moves.

Additionally, to prevent draw, when both the agent and enemy total token height is less than 5, the agent will be designed to prioritising more on moving towards the enemy and focusing on chances to eliminate an opponent stack. A threefold repetition penalty is also added, where the final score from the evaluation function is deducted when the same move is repeated three times, forcing the algorithm to choose a different move. This is to ensure that the game does not end in a tie due to threefold repetition. When we reach turn 220, the 1st property weight is double so that it can be more aggressive and finish the game faster.

Weight for eating an enemy token or removing enemy token from the board is higher compared to the rest of the properties, as this ensures one step closer to the winning goal. The final weights are based on a lot of trial and error manual tuning and making different weighting agent go against each other.

Other aspect: Algorithms Optimisation and Special Data Structure

Move Ordering for Alpha-Beta Pruning

Moving order was applied to enhance the effectiveness of alpha-beta pruning. Similar to the evaluation function, different ordering strategies were designed for the placement and play phase. To reduce computational overhead, move ordering is based on simplified heuristic, while more precise evaluation is computed in minimax search.

For the placement phase, moves are prioritised based on a precomputed heatmap that rewards central positions and penalises edge and corner positions. This avoids repeated computation and provides a fast approximation of positional strength.

For the play phase, actions are prioritised in this order: Eat action, Cascade action, and Move action. The Eat action is given the higher priority as it can immediately eliminate opponent tokens when they are adjacent to the enemy. Cascade actions are prioritised next, as they can remove opponent tokens by pushing them off the board without requiring direct adjacency, making them highly efficient tactical moves. Move actions are considered last, as they are primarily positional and do not directly reduce opponent material.

Within eat actions, preference is given to moves where the attacking stack has a greater advantage over the target, to reduce the likelihood of the attacker being immediately eliminated in a subsequent move. Additionally, capturing higher-value opponent stacks is rewarded, since removing stronger enemy pieces provides greater strategic benefit.

For cascade actions, preference is given to moves that minimise the risk of losing the agent's own tokens. Similarly, move actions are biased towards merging friendly stacks, as higher stacks provide increased stability and enhance the ability to perform both eat and cascade actions in subsequent moves.

Overall, these ordering strategies improves alpha-beta efficiency by expanding more promising move first, allowing earlier pruning of less optimal branches and improving the effective search depth under time constraints.

Iterative deepening and Transposition table

Iterative deepening search combined with a transposition table is used to support efficient decision making under strict time and space constraints. Iterative deepening search combines the advantages of both breadth-first search and depth-first search. It repeatedly performs depth-limited minimax search with alpha-beta pruning, starting from depth 1 and gradually increasing the depth limit until a timeout is reached.

Although each iteration is performed using depth-first search, the algorithm ensures that all nodes up to a given depth limit are fully explored before proceeding to a deeper level. This provides a breadth-first like property in terms of uniform depth coverage, meaning that all legal moves are evaluated to approximately same search depth. As a result, the agent can fairly compare alternative moves under time constraints.

A key advantage of this approach is, there is always a valid best move available from the deepest fully completed search iteration. For example, if the search is interrupted during depth 5 due to time limit, the algorithm will return the best move found at depth 4. This ensures that even under time constraints, the agent can still produce a reasonable and well-informed decisions. Unlike a standard depth-first minimax search with a timeout, where the search may be interrupted mid-way through a depth, potentially resulting in incomplete or unevenly evaluated branches, iterative deepening avoids this issue by guaranteeing that returned results come from a fully evaluated layer of the tree.

Another advantage is that iterative deepening maintains the space efficiency of depth first search, requiring memory only proportional to the maximum search depth rather than storing all nodes in the search frontier as in breadth-first search. Although this approach involves repeated exploration of shallower depths, the overhead is acceptable because most of the computation occurs at the deepest level, and alpha-beta pruning significantly reduces the number of nodes expanded at each iteration.

During the place phase, although the agent still applies iterative deepening with a timeout constraint (timeout = 3s), the maximum achievable depth is also bounded by the structure of the game. Since each player is allowed exactly 4 Place actions, the place phase consists of at most eight total Place actions. As the game progresses, the number of remaining legal placement actions decreases, causing the effective search depth of iterative deepening to shrink over time.

Searching beyond the remaining placement horizon would enter the subsequent Play phase, where the game dynamics and evaluation criteria are different. Therefore, deeper search beyond the placement stage is neither required nor appropriate, since the placement heuristic is for a static environment with no Move, Eat and Cascade actions are allowed. As a result, the search depth is reduced dynamically as the game progresses, starting from 8 and decreasing to 1. This improves efficiency by avoiding unnecessary computation and allowing the agent to return a valid result earlier within the timeout.

In contrast, during the Play phase, the game becomes dynamic and no longer has a fixed finite horizon. Thus, iterative deepening is primarily bounded by the time limit (timeout = 3s), and the agent searches as deeply as possible within the available computation time. On average, the search reaches approximately 4-ply depth, although this varies depending on the state of the board.

As the number of tokens on the board increases, the branching factor also increases due to the large number of possible actions. This results in a higher computation cost per depth level, reducing the maximum achievable search depth within the fixed time limit. As a result, the effective search depth decreases in more complex board states where there are more available moves to evaluate.

To further improve the efficiency, a transposition table is used to store previously evaluated board states, avoiding redundant computation when identical positions are reached through different move sequences. Each state is represented using Zobrist Hashing (GeeksforGeeks, 2016), a lightweight hashing algorithm that effectively encodes the board states using XOR operations. Each hashed state acts as a key in the table and is stored along with the depth at which the evaluation was computed, the evaluation score, and a score flag indicating whether the stored value is an exact score, a lower bound, or an upper bound.

Having a flag on the evaluated score is necessary because alpha-beta pruning may terminate search early, meaning that the stored value may represent only a bound rather than the exact minimax evaluation. By storing the correct bound type, the algorithm can safely reuse cached information without invalidating correctness. Additionally, the table stores the best move found for each position. When a stored entry with an exact score is retrieved, this move can be prioritised in move ordering by exploring it first in the minimax search.

Although the transposition table increases memory usage due to caching board states, this trade-off is justified by the reduction in redundant computation. This combination of iterative deepening and transposition tables significantly reduces the number of expanded nodes, improves pruning efficiency, and prioritise strong moves earlier in the search (see Table 1 and 2). When a previously visited states is encountered again at the same or a comparable depth, the stored evaluation score can be reused directly, preventing repeated exploration of identical subtrees and enabling deeper search within the same time limit hence improving the time complexity as mentioned before.

Performance evaluation

To evaluate the performance of the agent, two baseline agents were implemented; a random agent and a greedy search agent. The random agent generates a list of all legal moves and selects an action at random in both the Place and Play phase. This agent does not use any domain knowledge or heuristic function and serves as a simple baseline for validating that our strategy performs better than random.

In addition, a greedy search agent (1-ply agent) was developed to provide a stronger baseline. This agent evaluates all legal actions from the current state and selects the move with the highest heuristic score, without performing deeper search or considering long-term future consequences. During the Place phase, the greedy heuristic prioritises placing stacks near the centre, avoiding edges to reduce the risk of being pushed off the board and positioning pieces closer to the opponent while avoiding early eating in Play phase and considering potential friendly merges. During the Play phase, the heuristic function considers stack height, potential kills through Eat and Cascade, and risk such as being eaten by enemy or losing pieces due to Cascade action.

These baseline agents were used to compare performance across multiple games, helping verify that the full minimax-based agent consistently outperforms simpler strategies. In addition, during the evaluation, the implemented agent is tested as the second player (Blue) while the baseline or testing agent plays as Red. This setup is used to control for first-player advantage, as Red has a natural advantage by moving first and therefore tends to control the tempo of the game. By fixing the agent as Blue, the evaluation ensures a fairer comparison between strategies under consistent starting conditions. A script was developed for automate the competition among the agents, running ten games for each testing agent, while collecting the relevant performance data presented in Table 1 and Table 2.

To evaluate the effectiveness of the proposed search and evaluation framework, comparisons are conducted between the full agent and simpler baseline strategies under controlled experimental conditions. In particular, the performance of the agent is influenced not only by the heuristic function, but also by the efficiency of the underlying search process, including iterative deepening and the use of a transposition table. Therefore, in addition to win rate, analysis on computational efficiency in terms of nodes expanded and average move length is performed, as these metrics directly reflect how effectively the search space is being prune and how efficiently previously evaluated board states are reused during decision making.

Evaluated properties	Random algorithm	Greedy Algorithm
Win Rate	100%	100%
Average Nodes expanded for current agent	1139729.60	6782889.40
Average moves taken	26.2	60.0

Table 1: Statistics of current agent algorithm **with** Transposition Table and Iterative Deepening vs baseline agents (random algorithm and greedy algorithm)

Evaluated properties	Random algorithm	Greedy Algorithm
Win Rate	100%	100%
Average Nodes expanded for test agent	4197736.75	10206356.75
Average moves taken	28.5	51.1

Table 2: Statistics current agent algorithm **without** Transposition Table and Iterative Deepening with depth 4 vs baseline agents (random algorithm and greedy algorithm)

From Table 2, the agent using the current evaluation function with a fixed depth (depth = 4) minimax search achieves a 100% win rate against all testing agents. This suggests that the evaluation function is effective against simpler opponents by considering multiple board properties rather than relying on random or purely local decisions. It also indicates that the features incorporated into minimax evaluation are sufficient to consistently outperform these baseline strategies. However, win rate alone is not sufficient to fully assess performance, so additional metrics such as nodes expanded and average move length are also considered.

As part of the improvement, Iterative deepening and a transposition table were implemented. As shown in Table 1, the reduction in nodes expanded indicates that the transposition table improves search efficiency by caching previously evaluated board states, preventing agent from repeatedly recomputing identical board states. Iterative deepening also helps the agent manage time more effectively by searching progressively deeper when possible and keeping the best result from the deepest completed search. Therefore, these techniques improve the practical efficiency of the minimax search which allows the agent to make stronger decisions within the time limit.

However, Table 2 shows a lower average number of moves against the greedy algorithm compared to Table 1 this may be because the algorithm made more direct or aggressive decisions which makes the game finish faster. Nevertheless, this does not mean the test agent was more efficient as the average node expansion was much higher, which means that it requires more computation to select its moves.

Supporting work

A defensive testing agent was created to evaluate how well the current agent performs against an opponent that avoids direct confrontation. This agent mainly focuses on escaping by splitting itself into small tokens and performing Eat actions when possible. It was developed after several test runs revealed that the current agent sometimes failed to chase such opponents effectively and will repeat similar movements (e.g. perform Move action repeatedly in a circle or moving back and forth) without making any meaningful progress. This revealed a weakness of the agent in the endgame, where it did not consistently prioritise closing distance to the opponent. Ideally, the agent should actively move towards enemy tokens to be able to Eat or Cascade. However, this was not observed during the testing. As a result, this defensive agent was useful for developing and testing the explicit endgame behaviour within the Play phase, helping the current agent make more purposeful chasing and trapping decisions against the enemy.

One of the trial and error explored in the heuristic function, was a risk calculation function which estimated the risk of being eaten and pushed off the board through after each move. However, during the testing, the agent that has this risk function performed worse and lost several games against the version without it. This suggests that the risk calculation may have caused the agent to behave overly defensively, reducing its ability to pursue winning opportunities. In addition, the extra computation introduced by this function increased time complexity, potentially limiting the search depth achievable within the time constraint. Therefore, the risk function was removed from the final implementation.

Reference list:

GeeksforGeeks. (2016, October 10). Minimax Algorithm in Game Theory | Set 5 (Zobrist Hashing). Retrieved from
GeeksforGeeks website: <https://www.geeksforgeeks.org/dsa/minimax-algorithm-in-game-theory-set-5-zobrist-hashing/>